REVISION CHEAT SHEET:
SQL SERVER DATABASE STRUCTURE
FILES • SCHEMAS • SYSTEM DATABASES

Creation • files • options • schemas • system databases

SQL Server Database Creation, Files, Schemas, Options, and System Databases

These topics are closely related. First understand the complete structure:

EXAMPLE / FLOW

```
SQL Server Instance
v
Database
|
+-- Data Files
|     +-- Primary Data File (.mdf)
|     +-- Secondary Data Files (.ndf)
|
+-- Transaction Log File(s) (.ldf)
|
+-- Schemas
|     +-- dbo
|     +-- sales
|     +-- hr
|
+-- Database Objects
+-- Tables
+-- Views
+-- Procedures
+-- Functions
```

There are also special system databases maintained by SQL Server:

EXAMPLE / FLOW

```
master
model
msdb
tempdb
Resource
```

We will understand each concept separately.

Database Creation

A database is an organized collection of related data.

In SQL Server, the simplest way to create a database is:

```sql
CREATE DATABASE EmployeeDb;
GO
```

Here:

```
EmployeeDb
```

is the database name.

SQL Server creates the required physical database files automatically using default locations and settings.

After creating it:

```sql
USE EmployeeDb;
GO
```

Now SQL statements execute in the context of EmployeeDb.

You can verify the current database:

```sql
SELECT DB_NAME() AS CurrentDatabase;
```

Expected result:

```
CurrentDatabase
---------------
EmployeeDb
```

The CREATE DATABASE statement creates both the database and the underlying files required by SQL Server.

Database Name vs File Name

This is an important distinction.

Suppose we create:

EXAMPLE / FLOW

```
EmployeeDb
```

That is the database name.

Internally, SQL Server stores that database using physical files.

For example:

EXAMPLE / FLOW

```
EmployeeDb
|
+-- EmployeeDb.mdf
+-- EmployeeDb_log.ldf
```

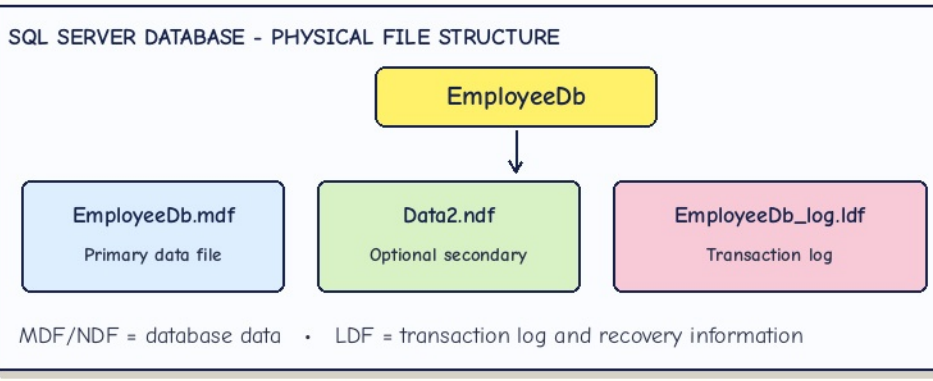
Therefore:

EXAMPLE / FLOW

```
EmployeeDb
=
Database name
EmployeeDb.mdf
=
Physical data file
EmployeeDb_log.ldf
=
Physical transaction log file
```

The database and the files are related, but they are not the same thing.

SQL Server Database Files



A SQL Server database needs files on disk to physically store its information.

There are two main categories:

EXAMPLE / FLOW

```
Database Files
|
```

EXAMPLE / FLOW (cont.)

```
+-- Data Files
|
+-- Transaction Log Files
```

Data files store database data and objects.

Transaction log files store transaction log information required for transaction processing and recovery.

Data Files

Data files contain the database's actual persisted data structures.

They can contain information related to:

EXAMPLE / FLOW

Tables
Indexes
Rows
Database objects
Other persisted database structures

SQL Server data files are normally divided into:

EXAMPLE / FLOW

Primary Data File
+
Optional Secondary Data Files

Primary Data File - MDF

Every SQL Server database has a primary data file.

The recommended extension is:

EXAMPLE / FLOW

.mdf

MDF is commonly understood as:



Master Data File / Primary Data File

For your learning, the important terminology is:



MDF is the primary data file of a SQL Server database.

Example:

EXAMPLE / FLOW

EmployeeDb.mdf

Conceptually:

EXAMPLE / FLOW

EmployeeDb
v
Primary Data File
v
EmployeeDb.mdf

The primary file contains database startup information and points to the other database files.

SQL Server's CREATE DATABASE examples use .mdf for the primary data file.

Does Every Database Need an MDF?

A normal SQL Server database has one primary data file.

Think:

EXAMPLE / FLOW

Database
v
Exactly one primary data file

For example:

EXAMPLE / FLOW

SalesDb
v
SalesDb.mdf

Even if the database has additional secondary files, it still has one primary data file.

Secondary Data File - NDF

A database can optionally have additional data files.

These are called:

Secondary data files

The recommended extension is:

EXAMPLE / FLOW

.ndf

Example:

EXAMPLE / FLOW

```
EmployeeDb_Data2.ndf
EmployeeDb_Data3.ndf
```

Conceptually:

EXAMPLE / FLOW

```
EmployeeDb
|
+-- EmployeeDb.mdf
+-- EmployeeDb_Data2.ndf
+-- EmployeeDb_Data3.ndf
```

The .ndf files are optional. Microsoft describes secondary files as user-defined optional data files that can be used to spread database data across multiple files or storage locations.

When Would We Use NDF Files?

For a small learning database:

EXAMPLE / FLOW

```
One MDF
+
One LDF
```

is usually enough.

Larger systems may use additional .ndf files for reasons such as:

EXAMPLE / FLOW

```
Large database size
Multiple storage devices
Filegroup design
Administrative requirements
Large-scale database architecture
```

Example:

EXAMPLE / FLOW

```
LargeSalesDb
|
+-- Sales_Primary.mdf
+-- Sales_Data01.ndf
+-- Sales_Data02.ndf
+-- Sales_Log.ldf
```

Do not create many .ndf files just because SQL Server supports them.

They should exist because there is an actual storage, administration, or performance requirement.

Transaction Log File - LDF

Every SQL Server database also requires a transaction log.

The recommended file extension is:

EXAMPLE / FLOW

.ldf

Example:

EXAMPLE / FLOW

EmployeeDb_log.ldf

Simple definition:

The transaction log records information about database changes so SQL Server can maintain transactional consistency and perform recovery.

For example:

SQL

```
UPDATE Employees
SET Salary = 80000
WHERE EmployeeId = 1;
```

SQL Server does not simply change the data file and forget what happened.

Information about database changes is recorded in the transaction log.

Conceptually:

EXAMPLE / FLOW

UPDATE
v
Transaction Log
v
Database change

Microsoft states that every SQL Server database must have at least one transaction log file.

Why Do We Need the Transaction Log?

The transaction log is extremely important for:

EXAMPLE / FLOW

Transaction rollback
Database recovery
Crash recovery
Backup/recovery operations

EXAMPLE / FLOW (cont.)

Maintaining transactional consistency

Suppose a transaction contains:

EXAMPLE / FLOW

Reduce Account A by $100
v
Add $100 to Account B

If the server fails during processing, SQL Server uses its transaction-management and recovery mechanisms, including the log, to determine what must be recovered appropriately.

You will study transaction log architecture separately later.

For now remember:



MDF/NDF store database data; LDF records transaction log information.



MDF vs NDF vs LDF

FileMeaningRequired?Main Purpose			
.mdf	Primary data file	Yes	Primary database data file
.ndf	Secondary data file	Optional	Additional database data storage
.ldf	Transaction log file	At least one	Transaction logging and recovery

Easy memory:

EXAMPLE / FLOW

MDF
=
Main/Primary Data
NDF
=
Additional Data
LDF
=
Transaction Log



Simple Database File Structure

A small database might look like:

EXAMPLE / FLOW

```
EmployeeDb
|
+-- EmployeeDb.mdf
|
|    v
|  Database Data
|
+-- EmployeeDb_log.ldf
v
Transaction Log
```

A larger database might look like:

EXAMPLE / FLOW

```
EmployeeDb
|
+-- EmployeeDb.mdf
+-- EmployeeDb_Data2.ndf
+-- EmployeeDb_Data3.ndf
|
+-- EmployeeDb_log.ldf
```

Logical File Name

This concept is commonly confusing.

Every SQL Server database file has:

EXAMPLE / FLOW

```
Logical File Name
+
Physical File Name
```

The logical file name is the name SQL Server uses internally to identify the file.

Example:

EXAMPLE / FLOW

```
EmployeeDb_Data
```

The physical file could be:

EXAMPLE / FLOW

```
D:\SQLData\EmployeeDb.mdf
```

So:

EXAMPLE / FLOW

Logical name
=
EmployeeDb_Data
Physical file
=
D:\SQLData\EmployeeDb.mdf

Database Name vs Logical File Name vs Physical File Name

Suppose:

EXAMPLE / FLOW

Database name:
EmployeeDb
Logical data file name:
EmployeeDb_Data
Physical data file:
D:\SQLData\EmployeeDb.mdf

These are three different things.

Think:

EXAMPLE / FLOW

EmployeeDb
v
Database
EmployeeDb_Data
v
Logical name used by SQL Server
D:\SQLData\EmployeeDb.mdf
v
Actual file on disk

Why Do We Need Logical File Names?

SQL Server commands frequently refer to files using their logical names rather than physical disk paths.

For example:

SQL

```
ALTER DATABASE EmployeeDb
MODIFY FILE
(
NAME = EmployeeDb_Data,
SIZE = 500MB
);
```

SQL (cont.)

Here:

EXAMPLE / FLOW

EmployeeDb_Data

is the logical file name.

SQL Server uses that logical identifier to find the corresponding database file.

Logical file names must be unique within the database.

How to See Logical and Physical File Names

SQL

Use:

SQL

```
USE EmployeeDb;
GO

SELECT
name AS LogicalFileName,
physical_name AS PhysicalFileName,
type_desc AS FileType
FROM sys.database_files;
GO
```

A result could look like:

EXAMPLE / FLOW

```
LogicalFileName      PhysicalFileName                  FileType
-------------------------------------------------------------------
EmployeeDb           C:\SQLData\EmployeeDb.mdf         ROWS
EmployeeDb_log       C:\SQLData\EmployeeDb_log.ldf     LOG
```

Notice:

EXAMPLE / FLOW

name
=
Logical name
physical_name
=
Actual location on disk

Creating a Database with Explicit File Settings

The simple version is:

```sql
CREATE DATABASE EmployeeDb;
```

SQL Server automatically chooses defaults.

But administrators can explicitly specify files.

Example:

```sql
CREATE DATABASE EmployeeDb
ON PRIMARY
(
NAME = EmployeeDb_Data,
FILENAME = 'D:\SQLData\EmployeeDb.mdf',
SIZE = 100MB,
MAXSIZE = 5GB,
FILEGROWTH = 100MB
)
LOG ON
(
NAME = EmployeeDb_Log,
FILENAME = 'E:\SQLLogs\EmployeeDb_log.ldf',
SIZE = 50MB,
MAXSIZE = 2GB,
FILEGROWTH = 50MB
);
GO
```

This uses the same structure documented for SQL Server's current CREATE DATABASE syntax.

The folders must already exist and the SQL Server service account must have the required access.

Understanding Important CREATE DATABASE Options

Look at:

```sql
NAME = EmployeeDb_Data
```

This is the:

EXAMPLE / FLOW

Logical file name

SQL

```
FILENAME = 'D:\SQLData\EmployeeDb.mdf'
```

This is the:

EXAMPLE / FLOW

Physical file location

SQL

```
SIZE = 100MB
```

This specifies the initial file size.

SQL

```
MAXSIZE = 5GB
```

This specifies how large the file is allowed to grow.

SQL

```
FILEGROWTH = 100MB
```

This tells SQL Server how much additional space to allocate when the file needs to grow.



File Growth

Suppose the file starts at:

EXAMPLE / FLOW

100 MB

and:

EXAMPLE / FLOW

FILEGROWTH = 100 MB

When SQL Server needs additional space:

EXAMPLE / FLOW

100 MB
v

EXAMPLE / FLOW (cont.)

200 MB
v
300 MB
v
400 MB

The file grows in 100 MB increments.

Autogrowth is a safety mechanism for additional required capacity.

It should not replace proper database capacity planning.

Fixed Growth vs Percentage Growth

SQL Server allows file growth to be configured using fixed sizes or percentages.

For example:

EXAMPLE / FLOW

100 MB growth

or:

EXAMPLE / FLOW

10% growth

For production systems, fixed-size growth is generally easier to reason about because each growth operation has a predictable size.

The appropriate size depends on the database workload and expected growth.

Data Files vs Log Files

This distinction is extremely important.

EXAMPLE / FLOW

Data File
v
Stores database data structures

EXAMPLE / FLOW

Log File
v
Records transaction log information

For example:

EXAMPLE / FLOW

EmployeeDb.mdf
=

EXAMPLE / FLOW (cont.)

```
Employee table/index data
EmployeeDb_log.ldf
=
Transaction log records
```

They serve different purposes.

Microsoft recommends considering separate storage for data and log files in production environments rather than automatically placing them on the same physical disk.

Filegroups - Basic Understanding

Although you did not ask to study filegroups deeply yet, you need one small idea because data files use them.

A filegroup is a logical grouping of SQL Server data files.

For example:

EXAMPLE / FLOW

```
PRIMARY Filegroup
|
+-- EmployeeDb.mdf
+-- EmployeeDb_Data2.ndf
```

Every database has a:

EXAMPLE / FLOW

```
PRIMARY
```

filegroup.

The primary .mdf file belongs to this filegroup.

Additional filegroups can also be created for advanced database storage design.

Transaction log files do not belong to filegroups.

For now remember only:

EXAMPLE / FLOW

```
Data files
-> belong to filegroups
Log files
-> do not
```

Database Options

A database option is a setting that controls some behavior of an individual database.

Simple definition:

Database options configure how a particular database behaves.

Each database can have its own options.

For example:

EXAMPLE / FLOW

EmployeeDb
-> one set of options
ShoppingDb
-> different options

Microsoft documents database-level options as settings unique to individual databases.

Where to See Database Options in SSMS

In SSMS:

EXAMPLE / FLOW

Object Explorer
v
Databases
v
EmployeeDb
v
Right-click
v
Properties
v
Options

There you can see many database-level settings.

Important Database Options

You do not need to memorize every database option now.

Important categories include:

EXAMPLE / FLOW

Recovery Model
Compatibility Level
Database State
User Access
Auto Create Statistics
Auto Update Statistics
Read/Write state
Page Verification
Query Store settings

Some of these deserve their own topics later.

Recovery Model

A database can use recovery models such as:

EXAMPLE / FLOW

```
SIMPLE
FULL
BULK_LOGGED
```

These affect transaction log management and backup/recovery capabilities.

Example:

SQL

```
ALTER DATABASE EmployeeDb
SET RECOVERY FULL;
GO
```

Do not select a recovery model randomly.

The correct choice depends on backup and recovery requirements.

We will study recovery models separately.

Compatibility Level

Compatibility level controls certain SQL Server Database Engine behaviors associated with SQL Server versions.

You can inspect it:

SQL

```
SELECT
name,
compatibility_level
FROM sys.databases
WHERE name = 'EmployeeDb';
```

Compatibility level is not the same thing as the SQL Server version itself.

We will study it separately.

AUTO_CREATE_STATISTICS

This option allows SQL Server to automatically create statistics useful for query optimization.

Example:

SQL

```
ALTER DATABASE EmployeeDb
SET AUTO_CREATE_STATISTICS ON;
```

SQL (cont.)

```
GO
```

For most normal databases, automatic statistics features are important for query optimization.



AUTO_UPDATE_STATISTICS

This allows SQL Server to automatically update statistics when appropriate.

SQL

```
ALTER DATABASE EmployeeDb
SET AUTO_UPDATE_STATISTICS ON;
GO
```

Again, detailed statistics behavior belongs to the performance topic later.



Checking Database Options

You can inspect common database settings using:

SQL

```
SELECT
name,
state_desc,
recovery_model_desc,
compatibility_level,
user_access_desc,
is_auto_create_stats_on,
is_auto_update_stats_on
FROM sys.databases
WHERE name = 'EmployeeDb';
```

Microsoft recommends sys.databases or DATABASEPROPERTYEX when retrieving database settings.



Changing Database Options

The basic syntax is:

SQL

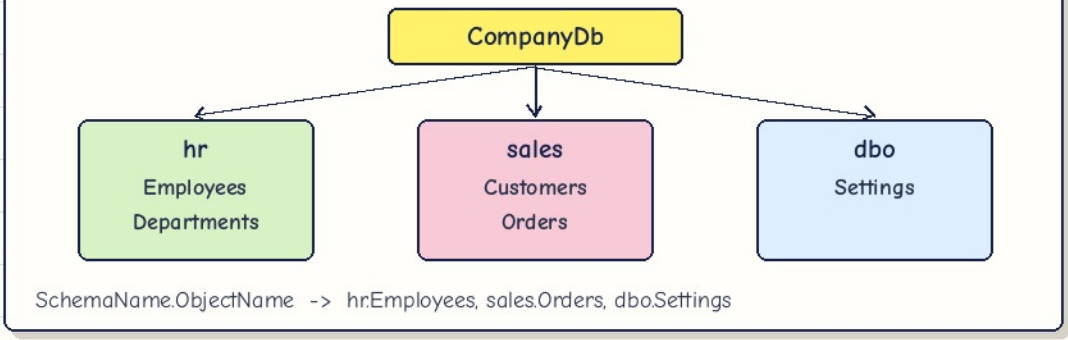
```
ALTER DATABASE DatabaseName
SET Option;
```

Example:

```sql
ALTER DATABASE EmployeeDb
SET AUTO_UPDATE_STATISTICS ON;
```

Some options can require special permissions or exclusive access, so database options should not be changed casually.

Schema



A schema is a logical container or namespace inside a database used to organize database objects.

Simple definition:

> A schema groups and names database objects such as tables, views, procedures, and functions.

Structure:

```
Database
v
Schema
v
Objects
```

For example:

```
EmployeeDb
|
+-- hr
|   +-- Employees
|   +-- Departments
|
```

EXAMPLE / FLOW (cont.)

```
+-- payroll
|   +-- Salaries
|   +-- Bonuses
|
+-- dbo
+-- Settings
```

Microsoft defines database schemas as namespaces or containers for objects such as tables, views, procedures, and functions.

Why Do We Need Schemas?

Without schemas, everything could be placed into one large collection:

EXAMPLE / FLOW

Employees
Departments
Orders
Customers
Invoices
Payments
AuditLogs

Schemas allow logical organization:

EXAMPLE / FLOW

hr.Employees
hr.Departments
sales.Orders
sales.Customers
billing.Invoices
billing.Payments

This improves:

EXAMPLE / FLOW

Organization
Object naming
Security management
Separation of business areas
Maintainability

Creating a Schema

Basic syntax:

```sql
CREATE SCHEMA SchemaName;
```

Example:

```sql
CREATE SCHEMA hr;
GO
```

Another:

```sql
CREATE SCHEMA sales;
GO
```

CREATE SCHEMA creates a schema in the current database.

Always make sure you are in the correct database:

```sql
USE EmployeeDb;
GO

CREATE SCHEMA hr;
GO
```

Creating an Object Inside a Schema

After creating:

```
hr
```

you can create:

```sql
CREATE TABLE hr.Employees
(
EmployeeId INT PRIMARY KEY,
FirstName VARCHAR(50) NOT NULL
);
```

Notice:

EXAMPLE / FLOW

hr.Employees

means:

EXAMPLE / FLOW

hr
=
Schema
Employees
=
Table

This is known as a two-part object name:

EXAMPLE / FLOW

SchemaName.ObjectName

Why Use Schema.Object?

Instead of:

```sql
SELECT *
FROM Employees;
```

prefer being explicit:

```sql
SELECT *
FROM hr.Employees;
```

Now SQL Server and developers clearly know which object is being referenced.

Same Table Name in Different Schemas

Schemas are namespaces.

Therefore, the same object name can exist in different schemas.

For example:

EXAMPLE / FLOW

sales.Customers
archive.Customers

These are two different tables.

Similarly:

EXAMPLE / FLOW

hr.Employees
audit.Employees

can be different objects.

This is one major reason schemas are useful.

dbo Schema

dbo stands for:

Database Owner

dbo is a built-in schema commonly present in SQL Server databases.

When beginners create tables without explicitly choosing another schema, they frequently end up in:

EXAMPLE / FLOW

dbo

For example:

SQL

```
CREATE TABLE dbo.Products
(
ProductId INT PRIMARY KEY,
ProductName VARCHAR(100)
);
```

The full object name is:

EXAMPLE / FLOW

dbo.Products

Here:

EXAMPLE / FLOW

dbo
=
Schema
Products
=
Table

Why Do We See dbo Everywhere?

If a user does not have another applicable default schema, SQL Server can fall back to dbo.

That is why beginner databases often look like:

EXAMPLE / FLOW

dbo.Customers
dbo.Orders
dbo.Products
dbo.Employees

Using dbo is not wrong.

For a small application, it may be completely acceptable.

For larger applications, business-oriented schemas can improve organization.

For example:

EXAMPLE / FLOW

sales.Customers
sales.Orders
inventory.Products
inventory.Stock
billing.Invoices
billing.Payments

dbo User vs dbo Schema

This is an advanced distinction worth remembering.

SQL Server has:

EXAMPLE / FLOW

dbo user

and:

EXAMPLE / FLOW

dbo schema

They are related historically, but they are not the exact same concept.

For your current level, remember:

dbo schema is a built-in schema used as a namespace for database objects.

We can study database ownership/security separately later.

Listing Schemas

You can see schemas with:

```sql
SELECT
name AS SchemaName
FROM sys.schemas
ORDER BY name;
```

sys.schemas contains one row for every database schema.

You may see:

```
dbo
guest
INFORMATION_SCHEMA
sys
hr
sales
```

Some are system-provided, while others may be user-created.

The sys Schema

You will often see objects such as:

```sql
sys.databases
sys.tables
sys.columns
sys.objects
sys.schemas
```

sys is a special system schema used to expose SQL Server catalog views and system metadata.

For example:

```sql
SELECT *
FROM sys.tables;
```

shows metadata about tables in the current database.

The underlying SQL Server system objects are physically stored in the Resource database but logically appear through the sys schema in databases.

Do not treat:

EXAMPLE / FLOW

sys

like your application's normal schema.

For application tables, prefer schemas such as:

EXAMPLE / FLOW

dbo
sales
hr
billing
inventory

INFORMATION_SCHEMA

You may also see:

EXAMPLE / FLOW

INFORMATION_SCHEMA

This is a special schema containing SQL-standard metadata views.

For example:

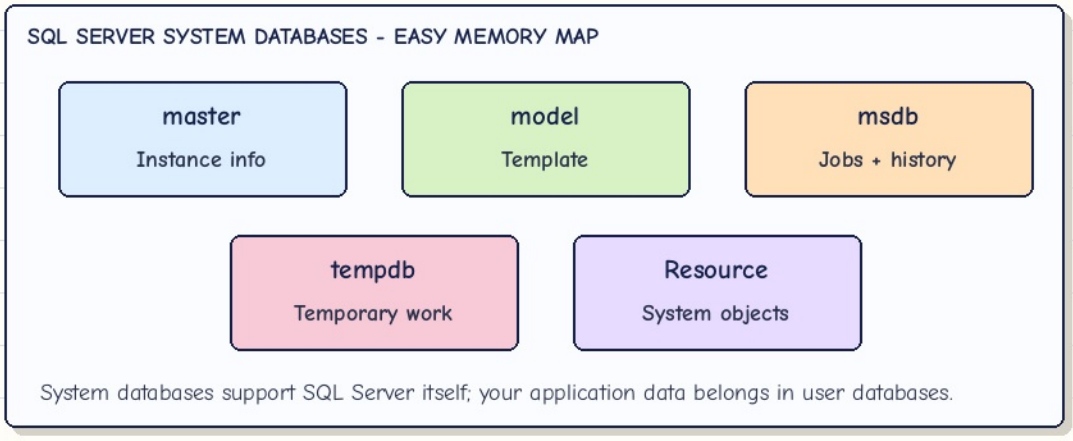
SQL

```
SELECT *
FROM INFORMATION_SCHEMA.TABLES;
```

It can provide information about database objects.

However, Microsoft notes that INFORMATION_SCHEMA exposes only a subset of SQL Server metadata, and SQL Server catalog views such as sys.objects are more complete for SQL Server-specific metadata.

Database System Databases



When you install SQL Server, SQL Server creates special databases used internally by the Database Engine.

These are called:

System databases

Important system databases include:

EXAMPLE / FLOW

master
model
msdb
tempdb
Resource

Your own databases are called:

EXAMPLE / FLOW

User databases

For example:

EXAMPLE / FLOW

EmployeeDb
ShoppingDb
FundraisingDb

master Database

The master database stores important SQL Server instance-level information.

Simple definition:

master contains essential information about the SQL Server instance.

Examples include metadata about:

EXAMPLE / FLOW

Databases
Database file locations
Server configuration
Logins
Linked servers
Endpoints

Most importantly, SQL Server records the existence and file locations of its databases in master.

SQL Server cannot start normally if the master database is unavailable.

Think:

EXAMPLE / FLOW

master
=
Main system-level information
about the SQL Server instance

Should We Create Application Tables in master?

No.

Do not create application tables such as:

EXAMPLE / FLOW

Customers
Orders
Products
Employees

inside master.

Microsoft specifically recommends not creating user objects in master.

Your application should use its own user database:

EXAMPLE / FLOW

EmployeeDb
ShoppingDb
FundraisingDb

model Database

The model database acts as a:

Template for newly created databases

When you create:

```sql
CREATE DATABASE EmployeeDb;
```

SQL Server uses model as part of the process of creating the new database.

Microsoft states that the contents and many settings of model are copied into newly created databases.

Think:

```
model
=
Template
```

Conceptually:

```
model
v
CREATE DATABASE EmployeeDb
v
EmployeeDb starts from model template
```

Changes to model can therefore affect subsequently created databases.

Do not casually modify it.

msdb Database

msdb supports SQL Server features related to administration and automation.

It is especially important for:

```
SQL Server Agent jobs
Job schedules
Alerts
Backup and restore history
Database Mail
Maintenance information
```

Microsoft specifically states that msdb is used by SQL Server Agent for scheduling jobs and alerts.

Think:

```
msdb
=
Jobs + administration history
```

For example, when you create a SQL Server Agent job:

EXAMPLE / FLOW

NightlyBackupJob

job information is maintained using msdb.

tempdb Database

tempdb is SQL Server's temporary working database.

Simple definition:

tempdb provides temporary workspace used by SQL Server and user operations.

It can be used for things such as:

EXAMPLE / FLOW

Temporary tables
Intermediate query results
Sorting operations
Hash operations
Versioning-related data
Internal SQL Server work

One of its most important characteristics is:

tempdb is recreated every time the SQL Server instance starts.

Its previous contents do not survive a SQL Server restart.

Think:

EXAMPLE / FLOW

tempdb
=
Temporary workspace

Example:

SQL

```
CREATE TABLE #Employees
(
EmployeeId INT,
Name VARCHAR(50)
);
```

A local temporary table like this uses tempdb storage.

Resource Database

The Resource database is a special read-only system database.

It contains SQL Server's built-in system objects.

For example, system objects exposed through:

EXAMPLE / FLOW

sys.objects
sys.tables
sys.columns

are physically associated with SQL Server's Resource database but logically appear in the sys schema of each database.

Its physical files include:

EXAMPLE / FLOW

mssqlsystemresource.mdf
mssqlsystemresource.ldf

You generally do not interact directly with the Resource database.

Think:

EXAMPLE / FLOW

Resource
=
SQL Server's built-in system objects

System Databases Comparison

System DatabaseMain Purpose	
master	SQL Server instance-level information
model	Template for new databases
msdb	Agent jobs, alerts, backup history and administration
tempdb	Temporary SQL Server workspace
Resource	Read-only built-in SQL Server system objects

Easy memory:

EXAMPLE / FLOW

master
=
Instance information
model

EXAMPLE / FLOW (cont.)

=
Template
msdb
=
Jobs and administration
tempdb
=
Temporary work
Resource
=
Built-in system objects

System Database vs User Database

System Database

Created and used by SQL Server itself.

Examples:

EXAMPLE / FLOW

master
model
msdb
tempdb

User Database

Created for your application.

Examples:

EXAMPLE / FLOW

EmployeeDb
ShoppingDb
FundraisingDb

Think:

EXAMPLE / FLOW

System database
=
SQL Server needs it
User database
=
Your application needs it

Complete Practical Example

Let's connect the concepts.

SQL

Create a database:

```sql
CREATE DATABASE CompanyDb;
GO
```

SQL

Use it:

```sql
USE CompanyDb;
GO
```

SQL

Create schemas:

```sql
CREATE SCHEMA hr;
GO

CREATE SCHEMA sales;
GO
```

SQL

Create tables:

```sql
CREATE TABLE hr.Employees
(
EmployeeId INT PRIMARY KEY,
FirstName VARCHAR(50) NOT NULL,
LastName VARCHAR(50) NOT NULL
);
GO

CREATE TABLE sales.Customers
```

SQL (cont.)

```
(
CustomerId INT PRIMARY KEY,
CustomerName VARCHAR(100) NOT NULL
);
GO
```

SQL

Insert data:

SQL

```
INSERT INTO hr.Employees
(
EmployeeId,
FirstName,
LastName
)
VALUES
(
1,
'John',
'Smith'
);

INSERT INTO sales.Customers
(
CustomerId,
CustomerName
)
VALUES
(
101,
'ABC Company'
);
GO
```

Now the logical structure is:

EXAMPLE / FLOW

```
CompanyDb
|
+-- hr
|    +-- Employees
|
+-- sales
+-- Customers
```

Physically, SQL Server also has files similar to:

EXAMPLE / FLOW

```
CompanyDb.mdf
CompanyDb_log.ldf
```

So there are two different views:

EXAMPLE / FLOW

```
Logical database view
---------------------
CompanyDb
v
Schemas
v
Tables
v
Rows
```

and:

EXAMPLE / FLOW

```
Physical storage view
---------------------
CompanyDb
v
MDF / NDF
+
LDF
```

That distinction is extremely important.

Common Mistakes

Thinking the Database Name and MDF Name Are the Same Thing

They may look similar, but they represent different things.

EXAMPLE / FLOW

```
EmployeeDb
=
Database
EmployeeDb.mdf
=
Physical file
```

Thinking MDF Is the Entire Database

A SQL Server database isn't simply one .mdf file.

It normally consists of:

EXAMPLE / FLOW

Data file(s)
+
Transaction log file(s)

Deleting MDF or LDF Files Manually

Do not manually delete database files from Windows while SQL Server is using them.

SQL Server manages these files.

Improper file manipulation can make a database unavailable or damage its recoverability.

Treating the LDF as Unimportant

The .ldf is not an optional temporary file.

The transaction log is a fundamental part of SQL Server database operation and recovery.

Creating Many NDF Files Without a Reason

Additional files increase administration complexity.

SQL

Use them when a storage/filegroup design actually requires them.

Putting Everything in dbo Automatically

dbo is perfectly valid, especially for small applications.

But larger systems can benefit from meaningful schemas:

EXAMPLE / FLOW

sales
hr
billing
inventory

Creating Application Objects in System Databases

Do not place normal application tables in:

EXAMPLE / FLOW

master
model
msdb
tempdb

SQL

Use your own user database.

Changing Database Options Without Understanding Them

A setting such as:

EXAMPLE / FLOW

Recovery Model
Compatibility Level
Auto Close
Statistics options

can affect behavior and performance.

Understand the requirement before changing production options.

Best Practices

SQL

Use meaningful database names:

EXAMPLE / FLOW

EmployeeDb
OrderManagementDb
FundraisingDb

SQL

Use meaningful logical file names:

EXAMPLE / FLOW

EmployeeDb_Data
EmployeeDb_Log

Plan file sizes instead of relying entirely on autogrowth.

SQL

Use sensible fixed autogrowth increments for production systems.

Monitor free disk space.

Do not manually manipulate active MDF/NDF/LDF files.

Keep application objects out of system databases.

SQL

Use meaningful schemas when they improve organization or security.

Reference objects using schema-qualified names:

```sql
SELECT *
FROM hr.Employees;
```

rather than relying unnecessarily on implicit schema resolution.

Back up important user databases and applicable system databases according to your recovery requirements.

Final Comparison

ConceptMeaning	
Database name	Name of the database, such as EmployeeDb
Logical file name	SQL Server's internal identifier for a database file
Physical file name	Actual file path on disk
MDF	Primary data file
NDF	Optional secondary data file
LDF	Transaction log file
Database option	Setting controlling database behavior
Schema	Namespace/container for database objects
dbo	Built-in commonly used schema
sys	System metadata schema
master	Instance-level system information
model	Template for new databases
msdb	Agent/jobs/admin history
tempdb	Temporary workspace
Resource	Built-in SQL Server system objects

Easy Memory Diagram

EXAMPLE / FLOW

```
SQL Server Instance
|
```

EXAMPLE / FLOW (cont.)

```
+-- System Databases
|      +-- master
|      +-- model
|      +-- msdb
|      +-- tempdb
|      +-- Resource
|
+-- User Database
|
+-- Data Files
|     +-- MDF
|     +-- NDF
|
+-- Log File
|     +-- LDF
|
+-- Database Options
|
+-- Schemas
+-- dbo
+-- hr
+-- sales
v
Tables
v
Rows + Columns
```

The simplest way to remember everything is:

EXAMPLE / FLOW

```
Database
=
Logical collection of application data
MDF / NDF
=
Where database data is physically stored
LDF
=
Transaction log
Schema
=
Logical organization of database objects
dbo
=
Common built-in schema
System Databases
=
Databases SQL Server itself needs
```

Important Interview Questions

1. What files make up a SQL Server database?

A SQL Server database normally contains one or more data files and at least one transaction log file.

EXAMPLE / FLOW

Data
-> MDF / NDF
Transaction Log
-> LDF

2. What is an MDF file?

An MDF is the primary data file of a SQL Server database.

A database normally has one primary data file.

3. What is an NDF file?

An NDF is an optional secondary data file used when additional database data files are required.

4. What is an LDF file?

An LDF is a transaction log file that records information required for transaction processing and database recovery.

5. Is an NDF file mandatory?

No.

A small database commonly has:

EXAMPLE / FLOW

One MDF
+
One LDF

without any NDF files.

6. Can a database have multiple LDF files?

SQL Server can support multiple log files, but adding log files isn't a normal performance-scaling technique. A database requires at least one.

7. What is a logical file name?

A logical file name is SQL Server's internal identifier for a database file.

Example:

EXAMPLE / FLOW

EmployeeDb_Data

8. What is the difference between logical and physical file names?

Logical:

EXAMPLE / FLOW

EmployeeDb_Data

Physical:

EXAMPLE / FLOW

D:\SQLData\EmployeeDb.mdf

The logical name is used by SQL Server commands; the physical name identifies the actual operating-system file.

9. What is a database schema?

A schema is a namespace or container used to organize database objects such as tables, views, procedures, and functions.

Example:

EXAMPLE / FLOW

hr.Employees

10. What does dbo.Employees mean?

EXAMPLE / FLOW

dbo
=
Schema
Employees
=
Table

Together:

EXAMPLE / FLOW

dbo.Employees

is the schema-qualified table name.

11. Why should schemas be used?

Schemas help with:

EXAMPLE / FLOW

Organization
Naming
Security
Maintainability
Separation of business areas

12. Can two schemas contain tables with the same name?

Yes.

For example:

EXAMPLE / FLOW

sales.Customers
archive.Customers

are different objects.

13. What is the master database?

master contains important instance-level SQL Server metadata, including information about databases and server configuration.

SQL Server depends on it to start correctly.

14. What is the model database?

model is the template SQL Server uses when creating new databases.

15. What is msdb?

msdb supports administration features such as SQL Server Agent jobs, alerts, backup/restore history, and related information.

16. What is tempdb?

tempdb is SQL Server's temporary workspace.

It is recreated every time the SQL Server instance starts.

17. What is the Resource database?

The Resource database is a read-only system database containing built-in SQL Server system objects.

18. What is the difference between a system database and user database?

System databases are maintained for SQL Server's operation.

User databases contain application/business information.

EXAMPLE / FLOW

master
=
System database
EmployeeDb
=
User database

19. What are database options?

Database options are database-level settings controlling behaviors such as recovery, compatibility, statistics, access, and other database characteristics.

20. What is the main difference between database files and schemas?

Files describe physical storage.

Schemas describe logical organization.

EXAMPLE / FLOW

MDF/NDF/LDF
=
Physical side
dbo/hr/sales
=
Logical organization

That distinction is one of the most important points from this topic.